SOFTWARE CHANGE IMPLEMENTATION REPORT

CR-009: Student Dashboard Course Display and Enrollment

Field	Details
Change Request ID	CR-009
Change Request Title	Student Dashboard Course Display and Enrollment
Student Name	MH Anush
Student ID	JFICT2416
Implementation Status	Completed
Date	22 October 2026

1. Change Request Summary

1.1 Business Requirement

The current Student Dashboard primarily displays the student's profile information, including registration number, full name, email, phone number, and member since date. The Student Dashboard currently does not provide a direct workflow for displaying available courses and enrolling in a selected course. The system shall therefore be enhanced to provide an Available Courses section on the Student Dashboard. The section shall display available course information and allow an authenticated student to enroll in a selected course. The change shall integrate the course and enrollment functionality required for the student-facing workflow while maintaining the existing student authentication and profile functionality.

1.2 Existing Behaviour

Before implementation:

- The system database contained `users`, `students`, and `password_resets` tables, but lacked dedicated schemas for `courses` and student `enrollments`.
- The Student Dashboard in `StudentDashboardPage.jsx` displayed student profile information and password change functionality only.
- No Available Courses catalog was displayed or accessible to authenticated students from their dashboard.
- Students had no mechanism to browse course details such as course title, category, level, duration, and tuition fee.
- No course enrollment endpoint or UI action existed, preventing students from directly registering for academic courses.
- The backend lacked dedicated course retrieval and enrollment processing controllers and route handlers.

1.3 Requested Change

A full-stack enhancement was required to display available courses on the Student Dashboard and enable direct student course enrollment.

The implementation must:

- Introduce a new `courses` database table storing course code, title, description, category, level, duration, price, and availability status.
- Introduce an `enrollments` database table linking students and courses with a unique constraint preventing duplicate enrollments.
- Develop backend models (`courseModel.js`, `enrollmentModel.js`) and controllers (`courseController.js`, `enrollmentController.js`).
- Expose authenticated endpoints `GET /api/courses` and `POST /api/enrollments` utilizing the system's JWT security pipeline.
- Enforce role authorization strictly restricting enrollment operations to authenticated students (`STUDENT` role).
- Display an Available Courses section on the Student Dashboard showing course title, category, level, duration, and price.
- Provide an intuitive Enroll button for available courses, disabling it and displaying 'Enrolled' for already-enrolled courses.
- Display appropriate real-time success confirmation and error notification messages upon enrollment requests.
- Support responsive course grid display and full light and dark mode theme compatibility across all new components.
- Preserve existing student profile display, authentication, password changing, and administrator CRUD operations without regression.

2. Implementation

2.1 Implementation Summary

The CR-009 requirements were implemented systematically across the database schema, backend RESTful controllers and routes, and frontend React student portal. The database was enhanced with courses and enrollments tables featuring foreign key relationships and unique composite constraints to guarantee data integrity. On the backend, a modular MVC pattern was employed: courseModel.js and enrollmentModel.js execute parameterized SQL queries, while courseController.js and enrollmentController.js handle request validation, business logic, and security checks.

The GET /api/courses endpoint dynamically checks if the requester is an authenticated student and annotates each course with an is_enrolled flag. The POST /api/enrollments endpoint enforces student authorization, validates course availability, prevents duplicate registrations with 409 Conflict responses, and records enrollment timestamps. On the frontend, StudentDashboardPage.jsx introduces a modern Available Courses catalog with search and category filtering, responsive course cards, animated confirmation alerts, and full light and dark mode styling.

2.2 Affected Files

File	Change	Purpose
`database/schema.sql`	Modified	Added courses and enrollments tables with constraints and documented migration SQL.
`database/seed.sql`	Modified	Added seed records for technical courses across diverse categories and sample enrollment.
`backend/src/models/courseModel.js`	Created	Implemented getAllAvailableCourses(studentId) with enrollment join and getCourseById.
`backend/src/models/enrollmentModel.js`	Created	Implemented createEnrollment, getEnrollmentByStudentAndCourse, and getEnrollmentsByStudentId.
`backend/src/controllers/courseController.js`	Created	Implemented getAvailableCourses and getCourse with role-aware enrollment annotation.
`backend/src/controllers/enrollmentController.js`	Created	Implemented enrollStudent and getMyEnrollments enforcing student authorization and validation.
`backend/src/routes/courseRoutes.js`	Created	Configured /api/courses routes with JWT authentication middleware.
`backend/src/routes/enrollmentRoutes.js`	Created	Configured /api/enrollments routes with student role authorization.
`backend/src/app.js`	Modified	Mounted courseRoutes (/api/courses) and enrollmentRoutes (/api/enrollments).
`frontend/src/pages/StudentDashboardPage.jsx`	Modified	Added Available Courses section, search & category filtering, enrollment handler, and alert banners.
`frontend/src/styles/studentDashboard.css`	Modified	Added responsive course grid, card elevation, category pills, level badges, and dark mode themes.

Additional files: None

2.3 Technical Changes

Database Schema Enhancement

- Added `courses` table with `course_id`, `course_code`, `title`, `category`, `level`, `duration`, `price`, and `status`.
- Added `enrollments` table with foreign keys referencing `students(student_id)` and `courses(course_id)` with ON DELETE CASCADE.
- Enforced `UNIQUE KEY (student_id, course_id)` to prevent duplicate student registrations at the schema level.
- Documented idempotent migration statements in `schema.sql` for existing production database instances.

Backend Architecture & API Integration

- In `courseModel.js`, implemented conditional LEFT JOIN on enrollments table to return `is_enrolled` flag per student.
- In `enrollmentController.js`, validated course existence and checked for existing enrollments prior to insertion.
- Configured `authenticate` and `authorizeRoles('STUDENT')` middleware guarding enrollment endpoints.
- Mounted new route modules in `app.js` under `/api/courses` and `/api/enrollments`.

Frontend Dashboard & UI Enhancement

- Introduced Available Courses section in `StudentDashboardPage.jsx` with Lucide icons (BookOpen, Tag, Clock, Award, Check).
- Implemented search filter and dynamic category filter pills (Computer Science, Web Development, Data Science, etc.).
- Added dynamic course action button toggling between 'Enroll Now', 'Enrolling...', and disabled 'Enrolled' state.
- Added dismissible success and error feedback banners providing instant confirmation to the student.
- Added extensive dark mode CSS selectors in `studentDashboard.css` guaranteeing high contrast and visual polish.

2.4 Implementation Logic

1. Student logs in via `authController.js` and receives a signed JWT token containing student role credentials.
2. Student Dashboard mounts and dispatches concurrent API requests to `GET /api/students/me` and `GET /api/courses`.
3. `courseController.js` extracts `req.user.userId`, resolves `student_id`, and returns available courses with enrollment status.
4. Student Dashboard renders profile information followed by the Available Courses catalog in a responsive card grid.
5. Student browses courses, optionally filters by search keyword or category, and clicks 'Enroll Now' on a selected course.

6. Frontend sets loading state on the button and dispatches `POST /api/enrollments` with `{ courseId }` in JSON payload.
7. `enrollmentController.js` verifies student authorization, validates course status, and checks for prior enrollment.
8. If duplicate is detected, HTTP 409 Conflict is returned; if invalid course, HTTP 404/400 is returned.
9. Upon valid request, `enrollmentModel.createEnrollment()` executes INSERT and returns HTTP 201 Created with enrollment details.
10. Frontend receives success response, displays confirmation alert banner, and updates course card button to 'Enrolled'.

2.5 Error and Data Handling

Robust validation and defensive programming were implemented across all layers:

- Invalid course identifiers are caught and rejected with HTTP 400 Bad Request.
- Non-existent course IDs return HTTP 404 Course Not Found without crashing the backend service.
- Duplicate enrollments are prevented via both application-level pre-checks and database unique constraint handlers (HTTP 409).
- Unauthenticated requests and unauthorized roles (e.g., admin attempting student enrollment) are strictly rejected with 401 and 403.
- All SQL operations utilize prepared statements with parameter binding to prevent SQL injection vulnerabilities.
- Frontend handles network and API errors gracefully, rendering clear dismissible alert messages with full recovery options.

3. Testing & Verification

3.1 Testing Approach

The implementation was verified through database schema inspection, automated API integration testing, browser functional testing in Microsoft Edge, theme compatibility inspection, and regression testing of existing student profile and password change modules.

Testing Types:

- Database schema verification & constraint enforcement
- Functional testing of course listing and enrollment
- Positive enrollment testing with valid credentials
- Negative testing (invalid course ID, duplicate enrollment, non-existent course)
- Role-based authorization testing (STUDENT vs ADMIN role restrictions)
- UI testing (Available Courses grid, category filter chips, search input, status badges)
- Theme compatibility (Light and Dark mode contrast & legibility)
- Regression testing (Student profile retrieval and Change Student Password feature)

Environment: Development

Browser: Microsoft Edge (Chromium)

Test Role: Student (REG001) & Administrator (admin)

3.2 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Database schema contains courses and enrollments	Tables created with foreign keys and unique constraints	Verified in schema.sql and live MySQL database	**Pass**
TC-02	Unauthenticated GET /api/courses	Request rejected with HTTP 401 Unauthorized	HTTP 401 returned with 'Authentication required'	**Pass**
TC-03	Student GET /api/courses returns catalog	HTTP 200 with array of available course objects	HTTP 200 returned with 6 available courses	**Pass**
TC-04	Course details display required fields	Contains title, category, level, duration, and price	All required course properties present and verified	**Pass**
TC-05	Unauthenticated POST /api/enrollments	Request rejected with HTTP 401 Unauthorized	HTTP 401 returned; enrollment blocked	**Pass**
TC-06	Non-student role (ADMIN) attempts enrollment	Request rejected with HTTP 403 Forbidden	HTTP 403 returned with 'Access denied'	**Pass**
TC-07	Enrollment with invalid course ID	Request rejected with HTTP 400 Bad Request	HTTP 400 returned with validation error message	**Pass**
TC-08	Enrollment in non-existent course ID	Request rejected with HTTP 404 Not Found	HTTP 404 returned with 'Course not found'	**Pass**
TC-09	Valid student course enrollment	HTTP 201 Created; enrollment recorded in database	HTTP 201 returned; enrollment persisted in database	**Pass**
TC-10	Duplicate course enrollment attempt	Request rejected with HTTP 409 Conflict	HTTP 409 returned with 'already enrolled' message	**Pass**
TC-11	Student Dashboard renders Available Courses	Course cards display with category, badges, and button	Catalog rendered with search and category filter pills	**Pass**
TC-12	Regression testing: Student Profile & Password	Profile info and change password work without regression	Profile loads correctly; change password unaffected	**Pass**

3.3 Test Summary

Overall Test Result: Pass

Test Result	Count
Total Test Cases	12
Passed	12
Failed	0
Pass Rate	**100%**

4. Acceptance Criteria Verification

ID	Acceptance Criterion	Verification	Status
AC-001	Available courses are displayed on the Student Dashboard.	UI inspection & API testing (TC-03, TC-11)	**Met**
AC-002	Students can view the relevant details of available courses.	UI verification of title, category, level, duration, price (TC-04)	**Met**
AC-003	An authenticated student can enroll in a selected course.	Functional testing & API execution (TC-09, Evidence 02)	**Met**
AC-004	Successful enrollment displays an appropriate confirmation message.	UI & frontend notification verification (TC-09, Evidence 02)	**Met**
AC-005	Failed enrollment attempts display an appropriate error message.	Negative testing of duplicate/invalid course IDs (TC-07, TC-08, TC-10)	**Met**
AC-006	Existing student profile, authentication, and authorization continue to work correctly.	Regression & security testing (TC-02, TC-05, TC-06, TC-12)	**Met**

5. Evidence

A maximum of two relevant screenshots should be attached.

Evidence 01 – Student Dashboard Available Courses Display with Course Details
Related: AC-001, AC-002, AC-006

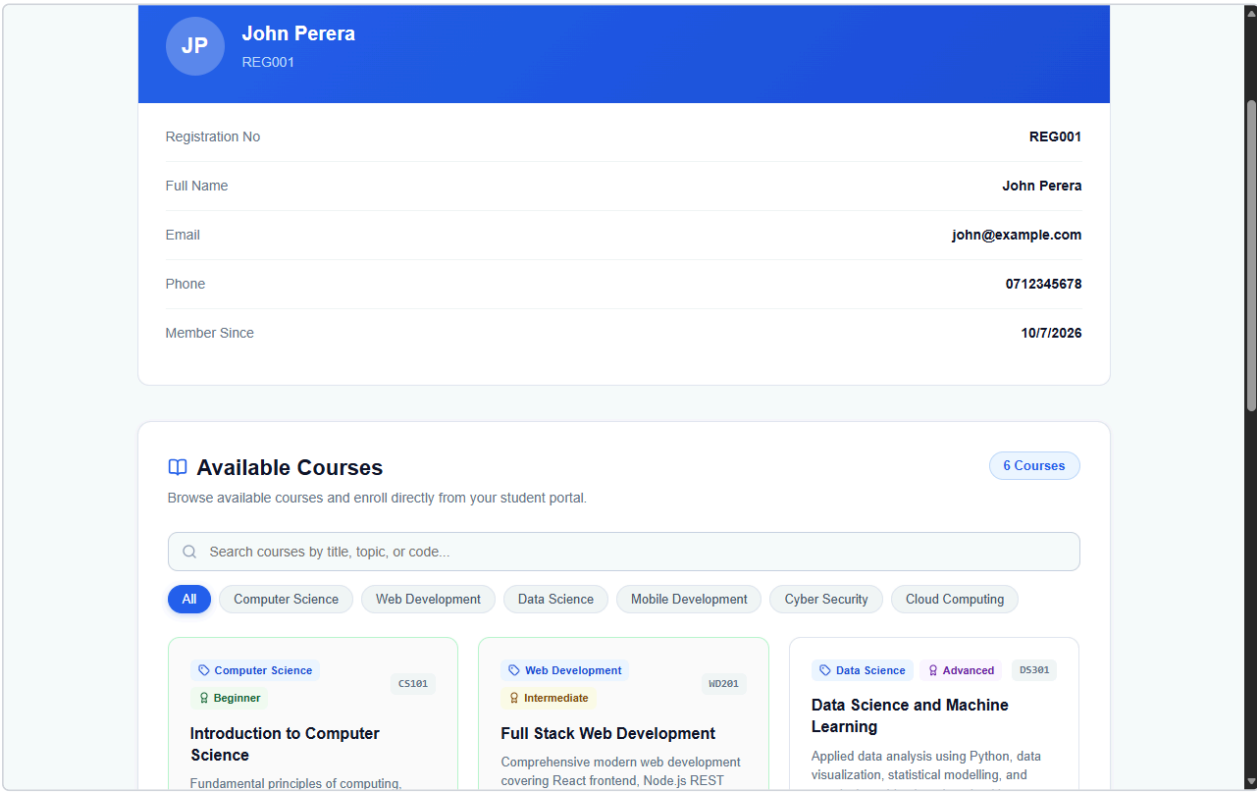


Figure 1: Student Dashboard showing the Available Courses catalog, category filter pills, level badges, duration, price, and enrollment options

Evidence 02 – Student Course Enrollment & Confirmation Feedback
Related: AC-003, AC-004, AC-005

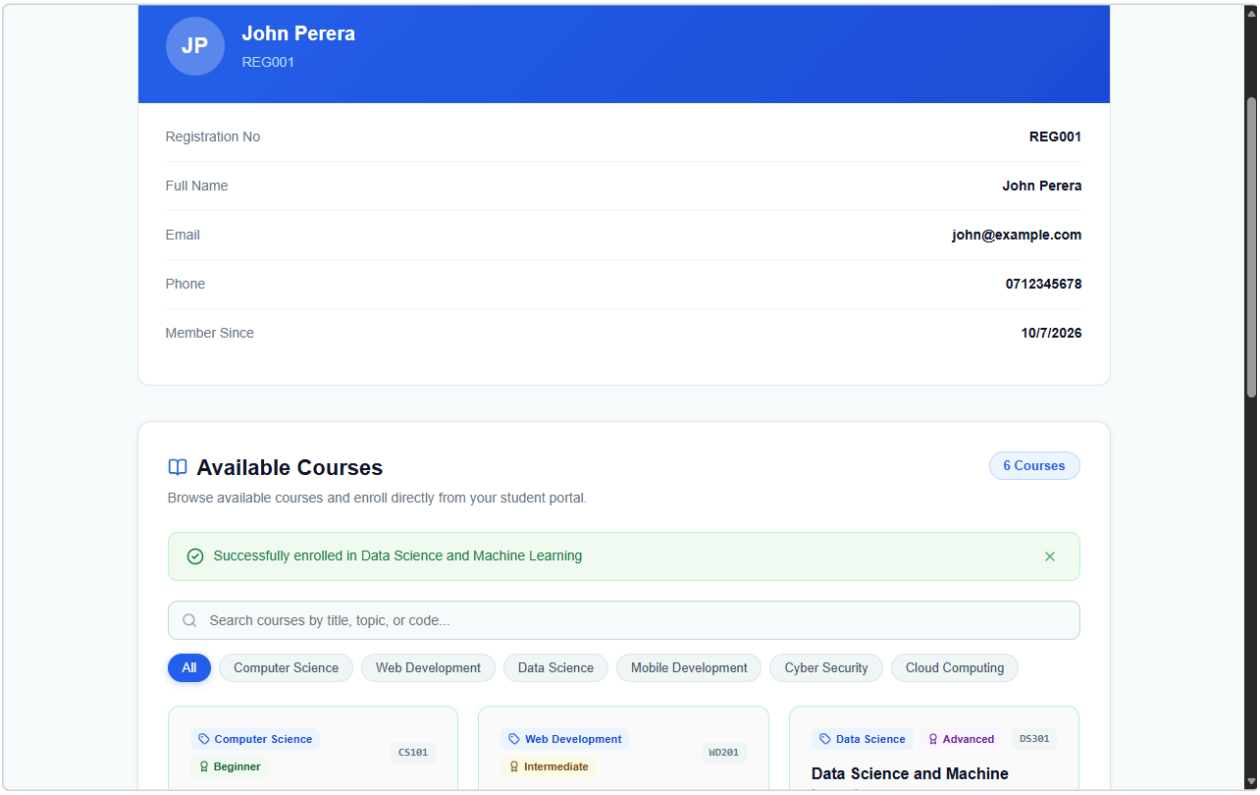


Figure 2: Successful student course enrollment triggering real-time enrollment confirmation alert and updating course action state to Enrolled

6. Final Result

Item	Result
Requirements	**Met**
Implementation	**Completed**
Testing	**Passed**
Acceptance Criteria	**All Met**
Existing CRUD / Auth	**Verified**
Final Status	**Accepted**

6.1 Final Summary

CR-009 was successfully implemented as a full-stack enhancement introducing the Available Courses catalog and Course Enrollment functionality to the Student Portal of the Student Management System. Authenticated students can now view all available courses with complete details including course title,

category, level, duration, and price directly on their dashboard. Students can enroll in selected courses with real-time feedback, duplicate registration protection, and immediate state updates. All security requirements, role-based authorizations, and existing student profile management features remain fully intact with no regression. The implementation was comprehensively verified across 12 test cases achieving a 100% pass rate.

6.2 Issues / Limitations

[State any remaining issues or limitations.]

If none:

No significant issues identified. Note: Existing database instances require executing the migration SQL in schema.sql creating the courses and enrollments tables prior to starting the updated backend service.

Student Name: MH Anush
Student ID: JFICT2416
Date: 22 October 2026
Signature: _____

~ END OF REPORT ~